情報拡散ライブビューア 企画・要件定義書

Oct 2, 2026 ・ @ソヨン

1. 背景と目的

本アプリは、最適系列を全探索で求める道具ではなく、1つのシナリオを「生中継」で再生して新しい現象を見つけるための探索用シミュレータである。見つけた仮説は、従来どおりRustのバッチ実験で統計的に検証する。

現状の課題

- 既存結果（内部行動促進→外部行動促進の順で外部行動者が約1.4〜3.8倍）は、理論から予想できる範囲にとどまり、新規性の面で弱い。
- 新たな変化を見るには、メッセージの入力方法（途中で変える、反応を見て選ぶ）と時間軸（Tラウンドの長期化）を広げる必要がある。
- しかしモンテカルロ全探索は系列数がメッセージ種類数Mの T 乗で増える。M=4 なら T=5 で1,024通り、T=20 で約1.1兆通りとなり、現実的に回らない。

本アプリで確認したいこと

1. 入力を固定系列からオンライン選択（前ラウンドの結果を見て選ぶ）に変えると、順序効果はどう変わるか。
2. Tを長くしたとき、外部行動の頭打ち・再加速・飽和がどのラウンドで起きるか。
3. IPの位置（中心性上位か周縁か）によって、拡散の広がり方とコンバージョンの出方がどう違うか。

位置づけ：今回は、現行の Rust シミュレーションを土台に、拡張機構 M1〜M5 を試作から適用した探索・可視化用シミュレータを作る。M6（適応方策）は保留する。本アプリ＝仮説の発見、Rust バッチ実験＝仮説の検証、という役割分担にする。

2. コンセプト：「拡散ライブビューア」

IPノードを選び、用意したメッセージを1ラウンドずつ「発信」すると、ネットワーク上で受信・シェア・外部行動が波のように広がる様子をその場で見られる。次のメッセージは、前のラウンドの結果を見てから選べる。

基本の操作の流れ

1. ネットワークを読み込む（`graph-lib` 形式のエッジリスト）。
2. ユーザ状態（反応性 w、閾値）の生成条件と、伝播確率を設定する。乱数シードも固定できる。
3. IPノードをクリック、または中心性の順位から選ぶ。

4. メッセージパレット（m0〜m3 など）から1つ選び、「発信」を押す。
5. カスケードがホップ単位で再生される。受信・シェア・外部行動がノードの色で変わる。
6. ラウンドの結果（新規シェア数、新規外部行動者数、効用の分布）がグラフに追加される。
7. 4〜6を繰り返す。途中で巻き戻して別のメッセージを試す（分岐）こともできる。
8. 系列と結果を記録し、Rust バッチ実験に渡せる形式で書き出す。

既存プログラムとの違い

観点	既存（Rust 全探索）	本アプリ
入力	固定系列を事前に全列挙	1ラウンドずつ選ぶ（結果を見てから）
試行	1系列あたり MC 数百回の平均	1回の実現を再生（必要なら MC 平均も表示）
出力	T ラウンド合計の2値のみ	ラウンド別・ホップ別・ノード別の状態
T	5（全探索の上限）	20〜50程度まで自由
目的	最適系列の確定	現象の観察と仮説の発見

3. 想定ユーザーと利用シーン

主な利用者は研究者本人で、次に指導教員と発表の聴衆である。

利用者	利用シーン	求めること
研究者本人	新しい仕組み（購入者の口コミ、摩耗など）を入れたときの挙動を手で試す	パラメータを変えてすぐ再実行できる。結果を記録できる
指導教員	ゼミで「この系列だとなぜこうなるか」を一緒に確認する	1画面で状況が分かる。巻き戻しと比較ができる
学会・発表の聴衆	発表中のデモ、または録画	専門外でも直感的に分かる色分け。動画として書き出せる

4. スコープ

本版は「M1〜M5 を適用した1シナリオを対話的に再生する道具」に絞る。最適化や大規模統計は既存の Rust バッチに任せる。

やること

- 現行の Rust モデルに M1〜M5 を重ねた、1ラウンドずつのメッセージ選択と発信、ホップ単位のアニメーション再生
- Web 版（WASM）とデスクトップ版（egui）の2つの形で提供（計算の窓口 `sim-api` と画面仕様は共通）
- 利用者が選んだグラフの読込。IP からのホップ距離でまとめた全体図から、拡大して展開
- ノードの活性（内部・外部・両方）の色分けと、単一ノード・ノード群の効用推移の表示
- ラウンド別の時系列グラフ、記録と書き出し
- 2つのシナリオ（設定や系列の違い）の並列比較

やらないこと（本版）

- M6（適応方策）の実装。保留し、定義は6.5に残す
- 系列の全探索・ベイズ最適化（Rust バッチ側で行う）
- 複数人の同時編集、ログイン、サーバでの結果共有
- 実 SNS への接続や実データの取得

5. 機能要件

優先度は MoSCoW（Must＝初版に必須、Should＝初版で入れたい、Could＝余力があれば、Won't＝今回は入れない）で示し、保留中のものは「保留」と記す。

ID	分類	機能	内容	優先度
F01	入力	グラフ読込	使うグラフは利用者が選ぶ。`graph-lib` のエッジリスト（txt/tsv/csv、gz 可、`#` コメント行）を読み、有向・無向と `transposed` を選べる	Must
F02	入力	ユーザ状態生成	現行の反応性（パレート分布、β・γ、scale＝1/(β·T)）を維持し、s̄ は 0.133 に合わせる。シードから全ノードの初期状態を作る	Must
F03	入力	メッセージパレット	試作は6個（m0〜m5、6.2）。M4 有効時は「発信しない」m∅ も選べる。空間を増やすときは設定と本書に明記する	Must
F04	入力	IP選択	全体図または拡大図でクリック、または次数・PageRank・VC の順位から選ぶ	Must
F05	実行	1ラウンド発信	選んだメッセージで1回のカスケードを実行し、ノード状態を更新する	Must

ID	分類	機能	内容	優先度
F06	表示	ホップ再生	カスケードをホップ単位で再生。再生・一時停止・速度変更・コマ送り	Must
F07	表示	ノード状態の色分け	未受信（白抜き）／受信のみ（灰）／内部活性（青の円）／外部活性（緑の四角）／両方活性（四角を左右半分ずつ青と緑に塗り分け）。形でも区別する（色覚多様性への配慮）。内部活性は「累積」と「このラウンドのみ」を切り替えられる	Must
F08	表示	ラウンド時系列	新規シェア数、新規外部行動者数、累積外部行動者数、内部・外部効用の平均と分位点をラウンドごとに描く	Must
F09	操作	巻き戻し・分岐	任意のラウンドに戻り、別のメッセージで続ける。分岐はツリーとして保持する	Should
F10	表示	ノード詳細（単一）	ノードを1つ選ぶと、内部・外部の効用の推移（1を超えても上限なしで表示）、受信したラウンドと受信元、活性になったラウンドを表示	Must
F19	表示	ノード群の詳細	活性の種類（内部のみ／外部のみ／両方）で分けたノード群の、内部・外部の効用の平均推移を種類ごとに重ねて表示（帯は10〜90%点）。リングの選択や範囲選択で作った群も同様に扱う	Must
F11	表示	全体図と拡大	大規模グラフは IP からのホップ距離でまとめた全体図（同心円のリング）から始め、選んだリングや領域を拡大して個別ノードに展開する。リングごとの受信・内部活性・外部活性の数を表示。縮小地図で現在地を示す	Must
F12	実行	MC 平均の併記	同じ系列を MC で N 回回し、平均と幅を時系列に重ねる（1回の実現が典型かを確認）	Should
F13	表示	2画面比較	2つのシナリオを同じシードで左右に並べて同時再生	Should
F14	出力	記録と書出し	系列・設定・ラウンド別結果を JSON と CSV（Parquet）で保存。バッチ実験用の設定ファイルも出す	Must

ID	分類	機能	内容	優先度
F15	出力	動画・画像書出し	再生を動画（WebM/GIF）、時系列をPNG/SVG で保存	Could
F16	実行	機構の切替（M1〜M5）	M1〜M5 の強さをスライダーで設定し、機構ごとにオン・オフできる。「現行（すべて無効）」と「既定」のプリセットを持つ。無効値にすると現行モデルに戻る	Must
F17	実行	適応方策	観測に応じて次のメッセージを自動で選ぶ（M6、保留）	Won't（保留）
F18	運用	共有・ログイン	複数人での結果共有	Won't

6. 拡散モデル仕様

本版は既存の Rust `core`（`diffusion.rs`・`user.rs`・`decision.rs`）のモデルに、拡張機構 M1〜M5 を重ねて使い、可視化と対話操作を足す。M1〜M5 は試作から適用し、強さを 0（または 1）にすると現行モデルに戻る。M6（適応方策）は保留し、定義だけをこの章に残す。

なお、M1〜M6 は「メッセージ空間」ではなく「モデルに足す仕組み」の番号である。メッセージ空間は 6.2 の6個を指す。

6.1 基本モデル（現行モデルの挙動）

- 1ラウンド＝IP が1つのメッセージを発信し、独立カスケードで伝わる。辺ごとに伝播確率で受信し、各ノードはそのラウンドで1回だけ受信する。
- 受信時に内部・外部の効用をそれぞれ更新する。

```
u' = w \cdot u + (1 - w) \cdot s
```

- 内部効用が正ならシェアし、次のホップへ伝える（毎回再評価、何度でも発火）。外部効用が正なら外部行動（生涯1回）。
- ラウンドをまたいで効用は持ち越し、受信フラグはラウンドごとにリセットする。
- 本版の入力は「1ラウンドずつ手で選ぶ」方式で、固定系列の再生もできる。T は 5 に縛られず、20〜50 程度まで伸ばせる。

6.2 メッセージ空間（試作は6個）

メッセージ空間とは「1ラウンドで発信できるメッセージの一覧」で、各メッセージは（内部刺激, 外部刺激）の組である。試作では、トレードオフ（内部と外部の合計が0.25）、中央一致（内部＝外部）、両方高を1つの空間に入れた次の6個を使う。

```
message_space = [
  { internal_stimulus = 0.20, external_stimulus = 0.20 },
  { internal_stimulus = 0.20, external_stimulus = 0.05 },
  { internal_stimulus = 0.15, external_stimulus = 0.10 },
  { internal_stimulus = 0.10, external_stimulus = 0.10 },
  { internal_stimulus = 0.10, external_stimulus = 0.15 },
  { internal_stimulus = 0.05, external_stimulus = 0.20 },
]
```

ID	内部刺激	外部刺激	型
m0	0.20	0.20	両方高
m1	0.20	0.05	トレードオフ（内部寄り）
m2	0.15	0.10	トレードオフ
m3	0.10	0.10	中央一致（内部＝外部）
m4	0.10	0.15	トレードオフ
m5	0.05	0.20	トレードオフ（外部寄り）

- m0 は他の5個すべてを上回り、m3 も m2・m4 に劣る。更新式は刺激について単調なので、M1〜M5 を無効（強さ 0）にした現行モデルでは、毎ラウンド m0 を発信するのが最良になると予想される（実行で確認する）。M2（摩耗）や M4（休止・忘却）などを有効にしたときに、この順位がどう崩れるかが見どころになる。
- M4 を有効にすると、6個に加えて「発信しない」m∅ を選べる。
- 平均刺激は両軸とも 0.8÷6≒0.133 である。
- メッセージ番号は外部刺激の昇順になっていない（m0 が最大、m1 が最小）。系列の「単調」判定などの分析では、外部刺激の順に並べ直してから使う。
- 他の空間（A11・B〜F など）は拡張するときに追加し、設定ファイルと本書にそのつど明記する。

6.3 反応性（現行のまま維持）

- 現行 config のパレート分布（β=0.8、γ=0.6、scale＝1/(β·T)）をそのまま使う。6個のメッセージの平均刺激は両軸とも約0.133なので、w の逆算に使う s̄ は 0.133 に合わせる。

- この較正では、予熱なしの1発で発火するノードは約3%、2発で約23%で、多くのノードはラウンド3〜4あたりで発火できるようになる。この性質も変えない。
- 実行 T を伸ばしても、較正に使う T は 5 に固定し、同じ集団を保つ（scale が T に依存するため、設定では「較正用 T」と「実行 T」を別の項目にする）。

6.4 アプリのために追加が必要な出力

現行の `diffuse` は2つの合計値しか返さない。アプリ用には、ホップごとに「どのノードが、誰から受信し、更新後の効用がいくつで、シェア・外部行動したか」のイベント列を返す関数（例：`diffuse_traced`）を追加する。既存関数と乱数の引き方を同じにして、同じシードで合計値が一致することをテストで保証する。

ノード詳細（F10・F19）のために、各ラウンドの終了時に全ノードの内部・外部の効用と、活性の種類（未受信・受信のみ・内部・外部・両方）を記録する。効用は上限を設けず、1を超えても丸めない。記録の大きさはノード数×ラウンド数×2値で、Twitter 規模（81,306ノード）・50ラウンドでも約32MB（f32）である。

6.5 試作から適用する機構（M1〜M5）と、保留する M6

すべて「強さを 0（または 1）にすると現行モデルに戻る」入れ子の形で足す。現行の結果を「強さ 0 の点」として自然につなげ、強さを連続的に上げたときに最適な型がどこで切り替わるか（境界）を示すためである。試作から M1〜M5 をすべて適用する。各機構の既定の強さは設計時に決める。M6 は保留する。

ID	名称	内容	式・パラメータ（現行に戻る値）	根拠	予想される変化	主な変更箇所
M1	購入者の口コミ（外部→内部の結合）	外部行動したノードは、その後の内部効用が上がり、拡散に参加しやすくなる	外部決定の成立時に内部効用へ κ を加える（κ=0）	採用者も情報を広める構造（Bhagat ら、WSDM 2012 の LT-C）	早めに転換させた購入者が拡散の担い手になり、外部寄りのメッセージを前のラウンドに置く系列が有利になる境界がありうる	`user.rs` の `contact`

ID	名称	内容	式・パラメータ（現行に戻る値）	根拠	予想される変化	主な変更箇所
M2	反復による摩耗	同じメッセージを続けて受けると効き目が落ちる	s_eff = s·ρ^(k−1)、k＝連続受信回数（ρ=1）	態度は露出回数に対して逆U字で、最大は約10回（Schmidt & Eisend、2015）	同じメッセージを連続して発信する系列が不利になり、メッセージを替える系列が有利になりうる。T=5 では効きにくく、長期化とセット	`user.rs`、`decision.rs`
M3	S字型の確率的な発火	効用が正なら必ず発火するのではなく、効用に応じた確率で発火する	P = 1/(1+exp(−u/σ))（σ→0）	S字型反応のもとではパルシングが最適になりうる（Management Science、2001）	「打つ・休む」を繰り返す配分や、外部促進を2回に分ける系列が有利になる領域	`decision.rs`
M4	休止・投稿予算・忘却	「何も投稿しない」ラウンド m∅、投稿回数の上限、受信がないラウンドで効用が初期値へ戻る	m∅ なし、予算なし、忘却率 0	パルシングと記憶の長さ（Aravindakshan & Naik、2015）	「間隔を空けるか、集中させるか」という問いが立つ	`message.rs`、`diffusion.rs`
M5	複合感染	同じラウンドに複数の隣人から経路が届くほど、刺激が強まる	s_eff = s·(1+η(n−1))、n＝届いた経路の数（上限 n_max）（η=0）	Centola & Macy（2007）	ネットワーク構造（クラスタリング）と系列の交互作用	`diffusion.rs`
M6	適応方策（閉ループ）【保留】	観測（シェアの伸び、転換者数）に応じて次のメッセージを自動で選ぶ	方策＝固定系列	多ラウンド影響最大化の適応版（Sun ら、KDD 2018）、適応的影響最大化	RAM と OPT の差（2〜14%）を、観測に基づく選択でどこまで埋められるか	アプリの制御ループ、`runner`

M4 の m∅ は「刺激 0 のメッセージ」とは別物である。刺激 0 のメッセージは受信のたびに効用を 0 に近づけるので予熱として働くが、m∅ は受信そのものが起きない。

6.6 M1〜M5 の既定の強さ

既定値は、現実の反応のしかたに近く、かつどの機構も結果を単独で支配しない大きさになるよう、バランスを取って置いた初期案である。画面では機構ごとにオン・オフでき、スライダーで強さを変えられる。

機構	パラメータ	既定	無効値	置いた理由
M1	κ：外部行動の成立時に内部効用へ加える値	0.05	0	刺激値の最小（0.05）と同じ大きさ。購入者の口コミは広告より弱いが、無視できない程度
M2	ρ：連続受信1回ごとに残る効き目の割合	0.90	1	5回連続の5回目で約0.66倍と穏やかで、T を伸ばすほど効く（10回目で約0.39倍）
M3	σ：S字の広がり	0.05	0（しきい値）	効用が ±0.05 のとき発火確率が約27%・73%。刺激値の最小と同じ尺度で、しきい値の近くだけをなだらかにする
M4	λ：受信のないラウンドで初期値へ戻る割合	0.10	0	5ラウンド受信がないと、効用の積み上がりの約4割が失われる（$0.9^5 \fallingdotseq 0.59$ が残る）
M4	B：投稿予算（発信できるラウンド数）	T の60%（T=5なら3回）	制限なし	毎ラウンド発信し続けない運用を想定した仮置き。m∅ で休止できる
M5	η：届いた経路が1本増えるごとの強化、n_max：数える経路の上限	η=0.25、n_max=3	η=0	2本届くと1.25倍、3本以上は1.5倍で頭打ち

- 既定値は文献の測定値ではなく、刺激値（0.05〜0.20）、効用の範囲、T=5〜20 との釣り合いから置いた仮置きである。P1 で確認して調整する。
- P1 では「すべて無効」「既定」「既定の2倍」の3水準で同じ系列を流し、各機構の効き方を見る。プリセットは「現行（すべて無効）」と「既定」の2つを持つ。

- 定義：k は、ノードごとに直前の受信と同じメッセージなら 1 増やし、違えば 1 に戻す。M2 は内部・外部の両方の刺激にかける。M1 の κ は、外部行動が成立した時点で1回だけ加える（外部行動は生涯1回のため）。M4 の忘却は、そのラウンドに受信しなかった全ノードで $u \leftarrow u + \lambda(u_0 - u)$（$u_0$ は初期効用）とする。
- M5 は、同じホップの処理が終わってからノードごとに n を数えて刺激を決める。乱数の引き方は現行と変えない設計とし、再現性は強さ 0 の一致テストで確認する。
- M3 を有効にしても、反応性の較正は現行のまま変えない（σ により初期集団の発火しやすさがわずかに変わる）。

7. 非機能要件

最も重い要件は「同じシードならバッチ実験と結果が一致する（再現性）」で、性能目標は Phase 1 の試作で実測して確定する。

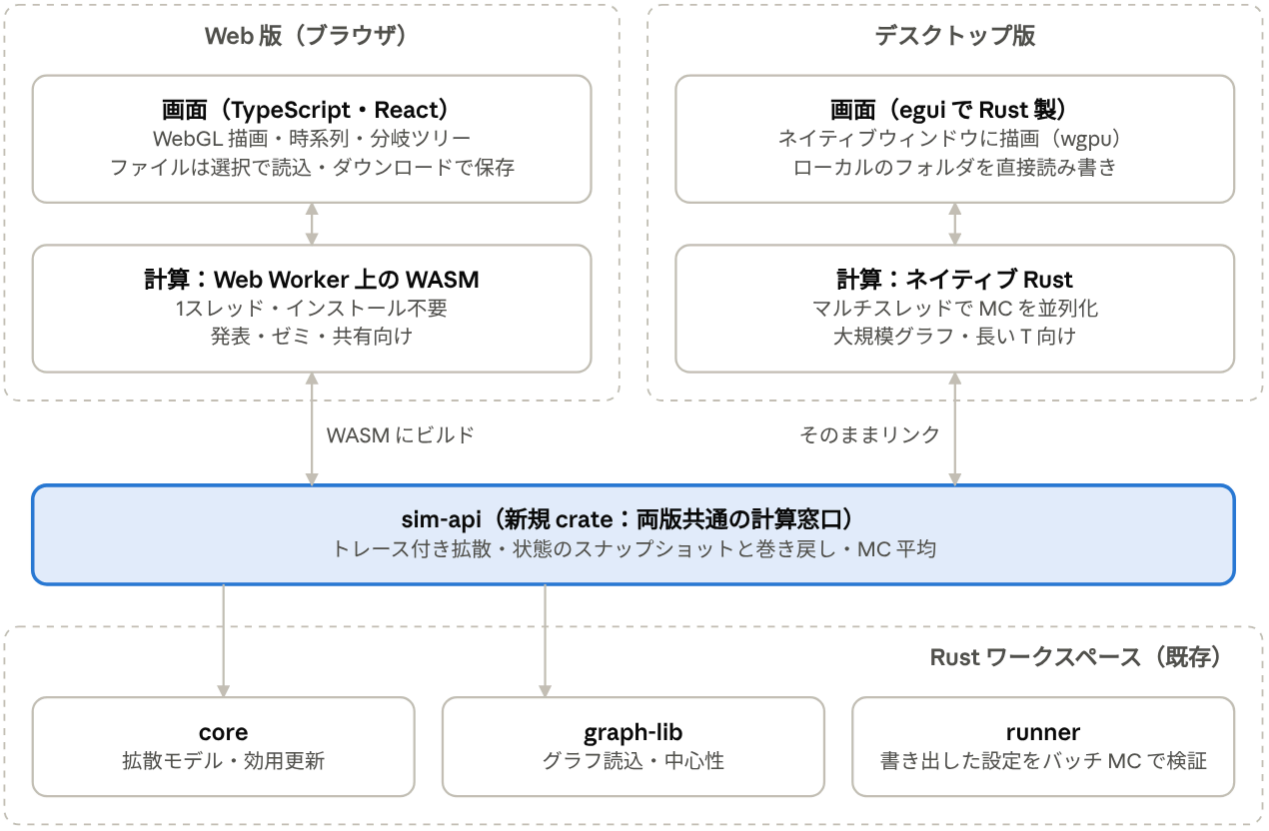
ID	項目	要件	備考
N01	再現性	同じグラフ・設定・シード・系列なら、Rust バッチの合計値とビット一致する	`core` を共有することで担保。自動テストで確認
N02	対話性能	1ラウンドの計算はクリックから 0.5 秒以内（Twitch 規模、約1万ノード）	計算は Rust、描画とは分離
N03	描画性能	1万ノード程度まで、全ノードを個別に色分けして滑らかに再生	WebGL 描画を前提
N04	大規模グラフ	Twitter（81,306ノード）級は全体図（IP からのホップ距離での集約表示）から始め、選んだ領域や IP 周辺を拡大して個別ノードに展開する	全ノード表示は Could
N05	動作環境	Web 版はブラウザ（Chrome / Edge 最新版）でインストールなしに動き、デスクトップ版は Windows・macOS で動く	発表会場でもオフライン動作
N06	データ保護	グラフや結果は外部サーバに送らない（すべてローカルで処理）	実 SNS データの扱いに配慮
N07	拡張性	新しい機構を `core` に1つ足すとき、UI 側はパラメータ定義を1か所追加するだけで済む	パラメータを宣言的に定義
N08	分かりやすさ	色だけに頼らず、形・凡例・数値でも状態を区別できる	専門外の聴衆、色覚多様性

ID	項目	要件	備考
N09	保守	`core` の単体テスト（既存）＋トレース版と既存版の一致テストを CI で回す	GitHub Actions 想定

8. システム構成：Web 版とデスクトップ版

Web 版とデスクトップ版の2つを作る。Web 版は「既存の Rust `core` を WebAssembly（WASM）にビルドし、ブラウザで動かす」構成、デスクトップ版は「同じ core をネイティブのまま呼ぶ」構成とする。両版とも計算の窓口（sim-api）と画面の仕様（9章）を共有する。画面は Web 版が TypeScript、デスクトップ版が Rust の egui で、それぞれ別に実装する。

Web 版とデスクトップ版は同じ計算窓口と core を共有する



システム構成·Web 版・デスクトップ版・共通の sim-api・Rust ワークスペース

モデルのコードは `core` の1つだけなので、どちらの版で見た結果も Rust バッチと一致する（N01）。

8.1 Web 版の設計

項目	内容
用途	ゼミ・発表でのデモ、指導教員との共有、インストールできない環境
計算	`sim-api` を WASM（`wasm-bindgen`）にビルドし、Web Worker で実行する。1スレッド
画面	TypeScript・React（Vite）。ネットワークは WebGL（sigma.js または Cosmograph）、時系列は uPlot など。両方活性の半分塗りノードは独自のノード描画で実装する
ファイル	ブラウザのファイル選択で読み込み、ダウンロードで保存する。外部サーバには送らない
規模の目安	1万ノード程度まで個別表示（Twitch 規模）。それ以上は全体図からの拡大で扱う
配布	静的ファイル一式。GitHub Pages またはローカルで開く。オフラインでも動く

8.2 デスクトップ版の設計

項目	内容
用途	大規模グラフ（Twitter 8万ノード級）、長い T、MC 平均を多数回回す確認
計算	`sim-api` を同じプロセス内で直接呼ぶ。MC は `rayon` で並列化し、全ノードの効用の記録もメモリ上で持つ
画面	Rust の egui（eframe）で実装する。ネットワークは wgpu による自前描画（8万ノードの一括描画を目標とし、P2 で実測）、時系列は egui_plot。Web 版とは画面の部品を共有せず、画面仕様（9章）だけを共有する
ファイル	ネイティブのファイルダイアログ（rfd）でローカルのフォルダを直接読み書きする。Parquet も Rust から直接扱え、既存の `result.parquet` と同じ場所に出力できる

項目	内容
規模の目安	IP からのホップ距離の全体図から始め、リングや IP の周辺を拡大して展開する（F11）
配布	Windows・macOS 向けの単一の実行ファイル（WASM や Node は不要）

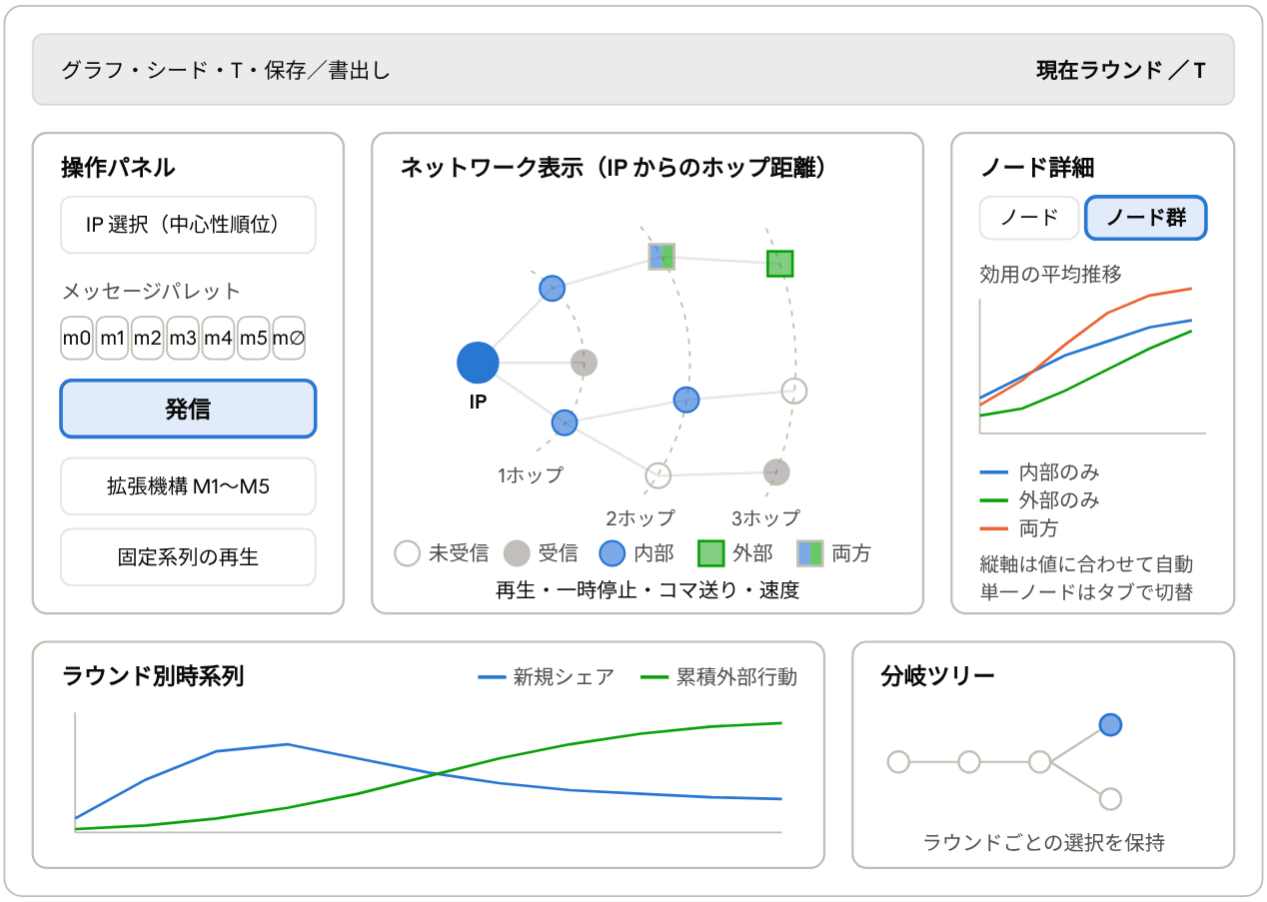
8.3 共通部分

- `sim-api`（新規 crate）：トレース付き拡散、状態のスナップショットと巻き戻し、全ノードの効用の記録、MC 平均、M1〜M5。両版でまったく同じコードを使う。
- 画面仕様：操作パネル、ネットワーク表示、ノード詳細、時系列、分岐ツリーの配置・色・操作を9章で定め、両版の共通の基準にする。実装は別（Web＝TypeScript・React、デスクトップ＝egui）。
- 計算の呼び出し口：Web＝Web Worker へのメッセージ、デスクトップ＝関数の直接呼び出し。
- レイアウトと中心性の事前計算：大きなグラフは Python（NetworkX・igraph）で CSV を作り、両版で読む。

9. 画面構成

画面は1枚で、左に操作、中央にネットワークの再生、右にノード詳細、下に時系列と分岐ツリーを置く。「発信→再生を見る→時系列で確認→次のメッセージを選ぶ」が視線の移動だけで回る配置にする。

操作・再生・結果を1画面に収める



メイン画面のワイヤーフレーム · 5領域

9.1 ノードの色分け（F07）

色だけでなく形でも区別し、色で区別しにくい人や白黒印刷でも読めるようにする。

状態	表示	判定
未受信	白抜きの円	まだメッセージが届いていない
受信のみ	灰色の円	届いたが、内部・外部とも活性になっていない
内部活性	青の円	内部の判定が成立した（シェアできる状態。後続がないノードでは実際のシェアは起きない）
外部活性	緑の四角	外部行動をした（生涯1回）
両方活性	四角を左右半分ずつ、左を青・右を緑に塗り分け	内部活性と外部活性の両方が成立した

内部活性は既定で累積（これまでに1回でも成立）として表示し、「このラウンドのみ」に切り替えられる。外部活性は生涯1回なので常に累積である。IP は大きな円で示す。

9.2 全体図：IP からのホップ距離（F11）

ネットワーク表示は IP からのホップ距離でまとめる。IP を中心に、1ホップ目、2ホップ目、…を同心円のリングとして並べ、リングごとの受信・内部活性・外部活性の数を表示する。大規模グラフでは最初にリング単位の全体図を出し、リングをクリックするとそのリングのノードを個別に展開する。IP から到達できないノードは「到達不能」の枠にまとめる。ネットワーク表示の隅に縮小地図を置き、いまどこを見ているかを示す。

9.3 ノード詳細（F10・F19）

ノード詳細には「単一ノード」と「ノード群」の2つのモードがあり、タブで切り替える。

モード	選び方	表示
単一ノード	ノードをクリック	内部・外部の効用の推移（折れ線）、受信したラウンドと受信元、活性になったラウンド
ノード群	活性の種類（内部のみ／外部のみ／両方）を選ぶ。リングの選択や範囲選択でも群を作れる	群ごとの内部・外部の効用の平均推移（帯は10〜90%点）、群のノード数の推移

- 効用は上限を設けず、1を超えても丸めない。縦軸は値の範囲に合わせて自動で広げる（M1の加算などで大きくなりうる）。

- 群は、表示中のラウンド時点の状態で分類し、そのメンバーを固定して過去のラウンドにさかのぼって平均する。「このラウンドで両方活性になったノードは、それ以前にどう推移したか」が見える。
- 2画面比較（F13）では、中央のネットワーク表示を左右に分割し、時系列は同じ軸に重ねる。

10. データ入出力仕様

入力は既存の `graph-lib` 形式と設定ファイルをそのまま使い、出力は「シナリオ記録（JSON）」と「ラウンド別結果（CSV / Parquet）」の2種類に分ける。

入力

種類	形式	内容
グラフ	txt / tsv / csv（gz 可）	1行1辺 `src, dst`。`#` で始まる行はコメント。txt はフォロー関係（A→B＝A が B をフォロー）なので `transposed=true` で拡散方向に反転する
レイアウト（任意）	CSV `node, x, y`	リング内の個別ノードの配置用に事前計算した座標。なければアプリ内で計算
中心性（任意）	CSV `node, degree, pagerank, vc`	IP 候補の順位表示用。なければアプリ内で計算
メッセージ空間	TOML（既存 `config.toml` 準拠）	メッセージ一覧 (内部刺激, 外部刺激) と、M1〜M5 の強さ（0 または 1 で無効）

ノード番号は `graph-lib` と同じく昇順ソート順で付ける（出現順ではない）。元の ID との対応表も保存する。

出力

種類	形式	主な項目
シナリオ記録	JSON	グラフのハッシュ、全パラメータ、シード、IP、分岐ツリー（各ラウンドのメッセージ）
ラウンド別結果	CSV / Parquet	round, msg, new_share, new_xact, cum_xact, mean_u_int, mean_u_ext, 分位点

種類	形式	主な項目
ノード別イベントと効用（書き出しは任意）	Parquet	round, hop, node, from, u_int, u_ext, kind（未受信・受信のみ・内部・外部・両方）
バッチ用設定	TOML	見つけた系列や方策を Rust バッチで MC 検証するための設定

メッセージ系列の文字列は「左端＝ラウンド1」に統一する（過去の LSD-first の混乱を避けるため）。

11. 開発フェーズとマイルストーン

本版は4段階で作り、各段階の終わりに「研究に使えるか」を判定する。M6 は保留し、必要になった時点で別途設計する。

フェーズ	内容	完了の判定	目安
P0 共通エンジン	`sim-api` crate（トレース版拡散、スナップショット、全ノード効用の記録）と、`core` への M1〜M5 の実装（強さ付き）。WASM とネイティブの両方でビルド	M1〜M5 を無効（強さ 0 または 1）にすると、既存版と同じシードで合計値が一致。両方のビルドが通る	3〜4週
P1 Web 最小版	小グラフで F01〜F08・F10・F14・F16・F19。ホップ距離の全体図、6個のメッセージ、M1〜M5 のスライダーとプリセット、両方活性の半分塗り、ノード詳細（単一・群）	「すべて無効」「既定」「既定の2倍」で同じ系列を流し比べられ、M1〜M5 の既定の強さを確認・調整できる	2〜4週
P2 デスクトップ版（egui）	egui で画面を実装し、9章の画面仕様に合わせる。ネイティブ計算と並列 MC。F11 のリング全体図と拡大、F19	Twitter 規模で全体図→拡大→発信ができ、Web 版と同じ設定・シードで同じ結果が出る	3〜4週
P3 研究機能・発表	F09 巻き戻し・分岐、F12 MC 併記、F13 2画面比較、F15 動画書出し	ゼミ・発表でデモができる	2〜3週
保留	M6 適応方策（F17）	必要になった時点で要件定義	—

目安の週数は研究・就活と並行する前提の仮置きで、P1 終了時に見直す。

12. リスクと未決事項

リスク

リスク	影響	対策
1回の実現は偶然に左右される	典型でない再生を見て誤った仮説を立てる	F13 で MC 平均と幅を常に併記。発見はバッチで必ず検証
アプリ版と Rust 版のモデルがずれる	結果が再現しない	`core` を共有し、一致テストを CI で回す（N01）
開発に時間を取られ、研究が進まない	修論・就活への影響	P1 で止めても使える範囲に絞る。UI の作り込みは後回し
大規模グラフの描画が重い	Twitter で使えない	計算と描画を分け、表示は集約。まず Twitch 規模で研究を進める
機構を足して「結論を作りにいった」と見られる	新規性の主張が弱まる	入れ子構造（強さ 0 で現行）と、強さを連続的に上げる提示

決定事項（2026-10-02）

項目	決定	反映先
今回の範囲	現行の Rust シミュレーションを土台に、M1〜M5 を試作から適用した探索・可視化用シミュレータを作る。M6 は保留	1・4・6章
技術構成	Web 版（Rust→WASM＋TypeScript）とデスクトップ版（Rust の egui）の両方を作り、それぞれ設計する	8章
グラフ	利用者が自分で設定する。大規模なら全体図から始めて拡大・展開する	5章 F11、7章 N04、9章
全体図	IP からのホップ距離でまとめる（同心円のリング）	9.2
ノードの色分け	内部活性は青の円、外部活性は緑の四角、両方活性は四角を左右半分ずつ青と緑に塗り分ける	5章 F07、9.1
ノード詳細	単一ノードと、活性の種類別のノード群の2モード。効用は1を超えても上限なしで表示	5章 F10・F19、9.3
メッセージ空間	試作は6個（トレードオフ4、中央一致1、両方高1）。増やすときはそのつど明記する	6.2

項目	決定	反映先
平均刺激	6個の平均は両軸とも約0.133で確定。反応性の β・γ・scale は現行のまま、s̄ は 0.133 に合わせる	6.2・6.3
拡張機構	M1〜M5 を試作から適用し、既定の強さは6.6の初期案。画面では機構ごとにオン・オフできる。M6 は保留	5章 F16、6.5・6.6

残る未決事項

- [ ] 試作に使う小グラフの選定（congress 475ノード、Twitch-ES 4,648ノードなど）
- [ ] 内部活性の既定の表示：累積（これまでに1回でも成立）か、このラウンドのみか（本書は累積を既定に仮置き）
- [ ] M1〜M5 の既定の強さ（6.6）は仮置き。P1 の3水準の流し比べで調整する
- [ ] egui での8万ノードの一括描画の性能は、P2 の試作で実測する